FRED TALKS

28th July 2026

CONTENTS

How not to forget what matters

HENRIK KARLSSON · 20 JUL 2026 · SOURCE

1.

Every few months I will read a tweet, or have a conversation, that makes me feel *this is important, I must remember this.* Often, these epiphanies are accompanied by a sense that *I actually know this already*, it had just somehow slipped my mind.

And for a few days, I do remember: my life shimmers with a new intensity, and I live the truth of what I grasped. But then, inevitably, the conveyor belt of things to pay attention to keeps churning, and my mind gets filled with small problems I need to solve, or new epiphanies or random noise, like news, and the shining fades from my eyes—I regress to being the same person as ever.

The Latin word for the tendency to lose track of what matters in the cacophony of things that attract our attention is *stultia*. "Stultia," writes Michel Foucault in "Self-writing,"

> is defined by mental agitation, distraction, change of opinions and wishes, and consequently weakness in the face of all the events that may occur; it is also characterized by the fact that it turns the mind toward the future, makes it interested in novel ideas, and prevents it from providing a fixed point for itself in the possession of acquired truth.

You can't just read a blog post about high agency, get filled with a sense of possibility, and become, from then on, an agentic person. As John Gray puts it in his monograph on J.S. Mill, our character is "a cluster of habitual willings." For changes to our behavior to become permanent, we must become different people.

In the same way that it is not enough to make a *resolution* that you will learn the piano, it is not enough to *realize* that when the kids act out, you shouldn't lose your temper but slow down, listen, and regulate their nervous systems with the help of yours. Imagine how good a person I would be if having insights were enough! But reacting to the frustrations of your children with calm and curiosity is a skill as much as playing the piano is—and as with the piano, the act of learning it requires rewiring your nervous system through sustained attention and practice. Realizing the value of acting in a certain way might give you a temporary motivation to do it. But in order to actually live in accordance with what you believe in long-term, you must make it a habit.

And this is much harder than making a habit out of playing the piano. When you're trying to make something like piano practice a habit, the standard advice is to chain it onto some already existing habit—to practice immediately after you brush your teeth in the morning, for example, or after you change out of your work clothes in the afternoon. Chaining the new habit to an already existing one provides a predictable trigger that helps remind you to practice. But the habits that make up our characters often do not follow a predictable schedule like this. I never know, for instance, when our children will act out (except that it will usually be when I'm least capable of handling it with grace—whenever both they and I are unusually hungry and tired). The conflicts seem to come out of nowhere, so I have to, somehow, *always* be ready to act in the proper way. I need to have the right reaction "ready at hand" (*procheiron*), as the Greek-philosopher-Roman-slave Epictetus put it. If Johanna and I talked about how we want to deal with the kids' conflicts the night before, I will nearly always handle the situation well. The problem is to keep it top of mind.

2.

During the first two centuries of the Roman Empire, there spread a practice known as *hypomnēmata*, a type of notetaking system, used as a tool for meditation,[1] in which the writer would store quotes from books they had read. Each day, often in the morning, the notetaker would open their notebook and look for a passage relevant to something they were struggling with, and then they would meditate on that—unpacking it, making the idea top of mind, ensuring it was alive in them. If they needed courage, for instance, they could meditate on an anecdote that made it real for them what it meant to act bravely. The idea was that over time, the insights they gathered by reading would be transformed into character, something deeply ingrained in their way of thinking and seeing and acting.

This was, as I understand it, an exercise designed to combat the problem I outlined above. Meditating on what matters is a simple habit, which you can chain onto your morning routine, but it reinforces the habits you can't plan, the habits that make up your character. It was, in the words of the French classicist Pierre Hadot, a spiritual exercise—an *exercise* because it required

work and discipline, *spiritual* because it engaged the whole person, not just their intellect, but their emotions and their moral character. It was an attempt to treat the formation of character as a skilled practice, as something you can deliberately train and improve through targeted exercises.

The most famous example of this practice is the meditations Marcus Aurelius wrote in his tent as plague swept through the camps during the military campaigns along the Danube River, but it seems to have been a fairly widespread practice among the "cultivated class." A suggestion repeated in several popular manuals for living was that you should collect every snippet of thought that deeply inspires you to live in a more ethically true way and then, in the pre-dawn hour, look through your hypomnēmata to find passages relevant to your current situation—insightful quotes, examples, actions you had witnessed, notes from conversations you'd had, and so on[2]—and meditate, in writing, on those that help you orient toward your current challenges, until you feel inspired to act in the proper way.[3]

What could be an example? Today, June 14, I woke up with a headache after having slept with my neck at an odd angle and so didn't feel like working. Having procrastinated for an hour or so, while Johanna tried to get me to start the day, I recalled principle 23 from a list Nabeel S. Qureshi has compiled for himself:

> Doing things is energizing, wasting time is depressing. You don't need that much 'rest'.

Walking around the farm with a cup of coffee,[4] reminding myself of the truth of this observation, I gained a sliver of motivation, enough to throw myself into rewriting this essay—and now my headache has lifted, and I feel excited. For more examples of things one might meditate on, I suggest looking at Nabeel's list. I also like to meditate on stories that make real to me some ethos I aspire to, such as the story of how Werner Herzog, dead broke while scouting for locations for *Nosferatu* in Brittany, happened upon a field of menhirs and decided to abandon his work to stay at the field for as long as necessary to solve the mystery of how the giant stones had been erected—a story that makes it visceral to me what it means to "take your curiosity seriously."

The hypomnēma was a mechanism for centering your mind on what matters, and for gradually refining your understanding of what matters. It was, to use a word from Plutarch, a tool for *ethopoiesis* (*ethos* meaning "character" and *poiesis* "making"), a tool for turning truth into character. By meditating daily on sentences that made it real to you how you wanted to live, you would remember to do the right thing—you would remember to practice it during the day (simply thinking about it is not going to change you). And gradually, you would become that

sort of person. You wouldn't even need to remind yourself. Your principles would have become your character. You would have developed expertise in the skill of holding yourself in a better way.

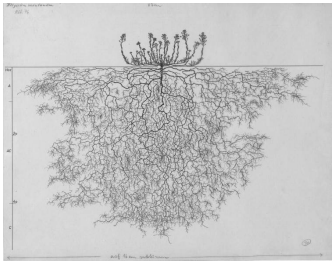
3.

I first came across the idea of using hypomnēmata in this way eight years ago when a friend sent me a copy of Foucault's "Self-writing." In the essay, Foucault talks about hypomnēmata and other modes of reading and writing used to fashion a self during the Hellenistic and Imperial eras. I didn't pick up the practice in its full form. But rereading the essay now, I realize how much it has influenced how I write: my essays are (often) meditations I do in order to deepen core ideas I want to live by, and to strengthen parts of myself I want strengthened.[5] (Writing treatises and letters was a common way of internalizing the content of the hypomnēmata.) And this practice has been transformative for me. I have often noticed that my experience of reality improves if I write and think about something.

But it strikes me now that the practice Foucault wrote about was probably more transformative than what I've ended up doing. Essay writing is incredibly time-consuming, and a lot of that time is spent on things that aren't self-transforming: I spend less time reshaping my mind than I spend solving literary-technical problems that help me write more functional and beautiful essays, for the joy of the craft and for the benefit of readers. Another limitation of my practice is that when an essay is done, I move on. The ideas—though they have been much deepened and more firmly lodged in my mind—fall out of attention and start to fade.

There is an element of self-deception involved here. I like to write essays, so it is comforting to think of it as a powerful practice, something that helps me live more fully and grow as a person. But if I look at it soberly, it is clear to me that essay writing is not a practice that is ideal for the purpose of ethopoiesis.[6] It is common to think that what we do achieves what we want it to achieve, even if there is no evidence for it. There are many practices that promise to transform and improve us—therapy, meditation, psychedelics—, but that branding doesn't mean that they actually do much for us: it is common to see people use these techniques for years without any obvious progress on their problems. If you want to achieve a particular outcome, it is important to start from that goal and evaluate which practices actually help you.

The most important ideas we need to return to weekly, even daily. Essay-writing, then, is not a functional substitute for having a practice that keeps the important truths top of mind, day after day. But it did help me reach that conclusion.



How to think in writing

Credits

This essay is based on a fragment from a longer essay on spiritual exercises that I wrote. That essay didn't work, and Johanna suggested using a few of the paragraphs for a new essay. She also suggested the main argument and general outline of the body of this essay, reorganized the second draft to shift the emphasis to what we came to see as the main idea, and corrected several faulty lines of argument I made.

The ideas were influenced by reading Pierre Hadot's Philosophy as a Way of Life *and Michel Foucault's* The Hermeneutics of the Subject, The Care of the Self*, and "Self-Writing." I was influenced by several conversations with Michael Nielsen and his notes on Tanya Luhrmann's* How God Becomes Real*, as well as a conversation with Alexander Obenauer about his practice. Claude Fable 5 (RIP) corrected four factual errors.*

1 [find in text]

The practice of keeping hypomnēmata had already existed for more than 400 years at this point, but the practices around it shifted significantly around this era, as I understand it, from a more conventional type of notetaking into the more specialized type of meditation described in this essay.

2 [find in text]

How do you actually keep these notes? A lot of people have insights and write them into Apple Notes, but then they never look at them again, and it becomes a mess after a while, so when they need the idea, it is nowhere to be found. I'm not sure exactly how the Romans managed to find their way back to the right wax tablet or papyrus. I guess the easiest thing is to just keep a single document for the most important ideas and stories, with short memorable concept handles (länk) that help you remember each. If you can't be bothered making your notes structured, though, LLMs are powerful enough now that they can trawl through your messy notes and find the

relevant quotes. Just give Claude Code or Codex access to your notes and ask it to find segments relevant to whatever you are thinking about. This will not be as powerful as doing the work yourself, but it is better than just forgetting.

3 [find in text]

I read somewhere that when meditating on their hypomnēmata, the Romans and Hellenistic Greek would also set intentions for themselves, principles they want to live by that day. I can't find the source for this claim now, but it resonated with something Johanna often tells me: it is much easier to hold the line if you've articulated your position to yourself beforehand. If, for example, I'm on a call with someone who wants to propose a project, I will often get caught up in their excitement, or feel uncomfortable setting the right boundaries—I often don't even know what the right boundaries are, because there is too much to consider for me to figure out my position in real time—and so I will say yes to things I shouldn't. But if I have thought things through beforehand, and have articulated to myself what I'm trying to achieve, and what my principles are, I can more easily see if this is in line with what I want, and I can clearly articulate my position. I wish I did this more often. Or at least remembered the principle to always ask for time to think things through on my own before committing.

4 [find in text]

Seneca and Epictetus would have insisted on making meditating in writing. The argument is that writing engages you more deeply, transforming what you have read "into tissue and blood" (in vivres et in sanguinem). Using a an oft-repeated metaphor, Seneca writes, "We should imitate bees, who wander and pluck from flowers suited for making honey, then arrange and distribute through the combs what they have brought in... We must digest what we have read; otherwise it passes into the memory, not into the mind." Food, he says, is a burden so long as it stays what it was; only when it changes into blood and tissue does it become strength. It might feel like I understand a topic without writing about it, but if I earnestly try to unpack my thinking in writing, my ignorance is revealed, and my thinking transformed.

5 [find in text]

I'm not sure how much of the shift in my writing came directly from reading Foucault: it has been something that I've iterated myself toward. I've gradually written more with the intention of shaping my character and my attention, and this has been driven by the experience of deepened presence this has afforded me. But I'm pretty sure the ideas from that essay provided a template that helped me make sense and direct the evolution of my writing. I know I have often returned to the essay, and have used Plutarch's term ethopoetical writing to make sense to what I'm trying to do.

6 [find in text]

What it can do well, though, is to push deep into an idea and to restructure your thoughts about something. It is like an intervention where you want to shake things up properly, a three week session where you think and think and think about the same thing all the time. It is, I think, a useful complement to a daily practice.

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · SOURCE

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

> *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentically-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
□□□□□□□ Failed Checks: 1 □□□□□□□

FAIL  src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:
- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:
```

```
- remove duplicated rendering paths or dead production code
- move repeated styling or static configuration out of lazy-loaded Ja
- lazy-load rarely used flows instead of loading them on the main pat
(more examples here)

Do not fix this by:
- omitting requested scope or deferring needed UI work
- removing aria-label/title text or visible affordances
- deleting helpful empty/error-state copy
(more examples here)

For intentional growth, (procedure here) and explain why in the PR.
```

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

```
Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica
```

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · SOURCE

It’s a common position among software engineers that big egos have no place in tech[1]. This is understandable - we’ve all worked with some insufferably overconfident engineers who needed their egos checked - but I don’t think it’s correct. In fact, I don’t know if it’s possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it’s more complicated than “big egos make good engineers”. The most effective engineers I’ve worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What’s going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There’s a reason this joke is so popular:

The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added[2].

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech

companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why *really* effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout[3]. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled[4].

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization[5].

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

   ↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

   ↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

   ↩

4. It's more or less exactly this scene from Silicon Valley.

   ↩

5. This description sounds a bit sociopathic to me. But, on reflection, it's fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

   ↩